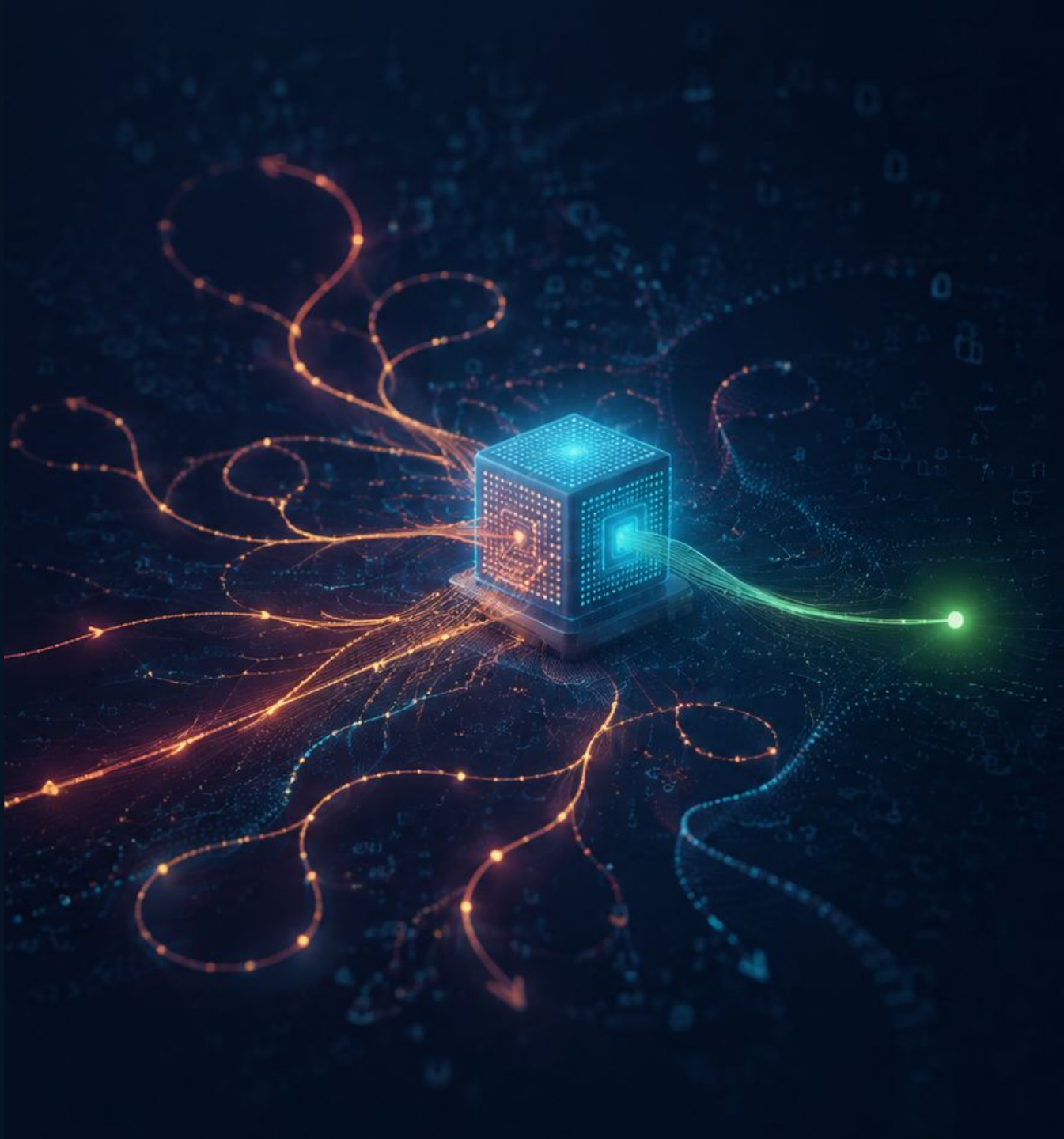


양자화된 추론 모델은 더 오래 생각해야 한다고 믿지만, 실제로는 그렇지 않다

Quantized Reasoning Models Think They Need
to Think Longer, but They Do Not

학생을 위한 논문 해설 · 40장

Sanae Lotfi · Polina Kirichenko · Steven Li · Zechun Liu
FAIR at Meta · Meta AI · arXiv:2606.00206 · 2026



첫 문제: 가장 큰 8자리 이진수는 ?

가장 큰 8자리 base-2 정수를 base-10으로 바꾸세요

$$11111111_2 = ?_{10}$$

생각할 시간 20초

- 8자리 모두 1이면 가장 큼니다
- 계산은 잠시 뒤에 공개합니다

정답은 255

$$11111111_2 = 2^8 - 1 = 255$$

$$11111111_2 = 2^8 - 1 = 255$$

- 정답을 찾는 데 긴 추론이 필요하지 않습니다
- 문제는 모델이 255를 찾은 뒤에도 계속 의심했다는 점입니다

논문의 핵심 한 문장

강한 양자화는 모델이 정답을 찾지 못하게만 하는 것이 아니라, 찾은 정답을 버리고 계속 생각하게 만든다

강한 양자화는 모델이 정답을 찾지 못하게만 하는 것이 아니라, 찾은 정답을 버리고 계속 생각하게 만든다

최대 52%

양자화 모델 실패 중 overthinking
CoT에서는 이미 정답을 한 번
찾았는데, 이후 계속 생각하다가
최종 답을 틀리게 낸 경우가 최대
52%

12–23%

penalty 적용 후 CoT 감소
(logit penalty
wait,but 이런 데에)

최대 58%

overthinking 오류 감소
정답을 찾고도 계속 생각하다
틀리는 overthinking error' 자체가
최대 58% 감소했다

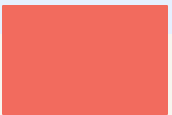
왜 양자화를 쓰는가

큰 모델을 더 적은 메모리와 비용으로 실행하기 위해 숫자의 정밀도를 줄인다

16-bit



3-bit

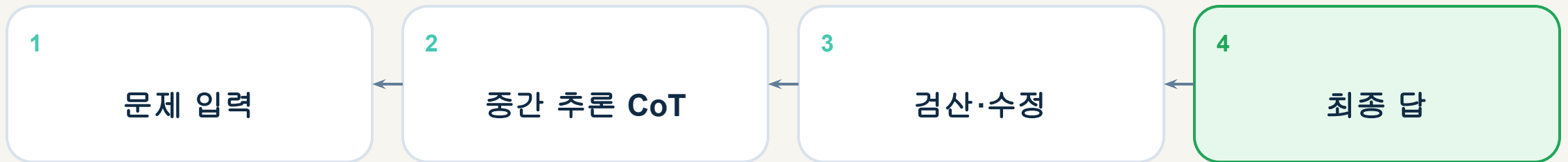


더 적은 메모리

- 배포 비용과 메모리 사용을 줄일 수 있음
- PTQ는 재학습 없이 적용 가능
- 그러나 추론 과정의 행동 변화는 덜 연구됨

추론 모델이 답을 만드는 과정

질문을 받은 뒤 중간 추론을 생성하고 최종 답을 출력한다



Chain-of-Thought의 장점과 위험

생각을 더 쓰면 복잡한 문제를 풀 수 있지만, 언제나 정확해지는 것은 아니다

도움이 되는 경우

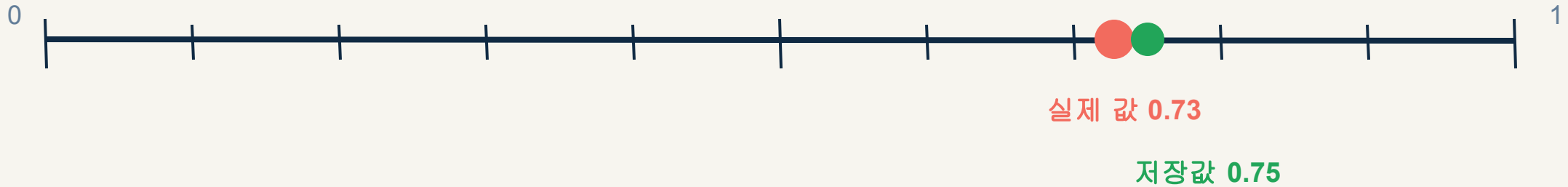
- 여러 단계를 연결
- 중간 계산을 확인
- 잘못된 경로를 수정

해가 되는 경우

- 이미 맞은 답을 의심
- 같은 계산을 반복
- 문맥 한도까지 계속 생성

양자화란 무엇인가

연속적인 값을 더 적은 수의 단계로 근사하는 압축 방법

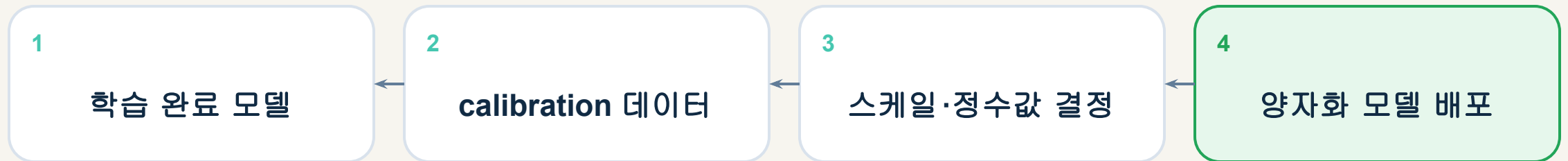


- 원래 값 0.73을 가까운 대표값 0.75로 저장
- 각 숫자의 오차는 작아 보여도 모델 전체에는 많은 오차가 쌓임
- 저장 공간은 줄지만 출력 확률이 조금씩 바뀔 수 있음

Post-Training Quantization

학습이 끝난 모델을 작은 calibration 데이터로 적정 scale을 찾아 quantization 한다.

AWQ -> calibration



논문이 비교한 세 가지 PTQ 방법

같은 비트 수라도 어떤 값을 보호하고 어떻게 변환하느냐가 다르다

방법	양자화 대상	핵심 아이디어	설정
GPTQ	가중치 중심	2차 정보로 layer별 오차 보정	3·4-bit
AWQ	가중치 중심	중요 activation channel 보호	3·4-bit
FlatQuant	W·A·KV	회전·스케일로 분포를 평탄화	W4A4KV4 / W8A8KV8

연구 질문

정확도 하락 뒤에서 실제로 어떤 추론 행동이 바뀌는가

01 양자화가 정확도와 CoT 길이를 어떻게 바꾸는가?

02 틀린 답은 능력 부족인가, 정답을 찾기도 버린 것인가?

03 원인을 겨냥한 간단한 추론 시점 개입으로 줄일 수 있는가?

실험 규모

모델 크기, 양자화 방식, 문제 영역을 넓게 교차 비교했다

5

추론 모델

3

PTQ 계열

5

벤치마크

1.5B–32B 모델 크기 · 수학·코딩·과학 · 28 model–quantization pairs

평가 모델: 1.5B부터 32B까지

작은 distilled model과 더 큰 reasoning model을 함께 평가



1.5B	DeepSeek-R1-Distill-Qwen
7B	DeepSeek-R1-Distill-Qwen
8B	DeepSeek-R1-Distill-Llama
14B	DeepSeek-R1-Distill-Qwen
32B	QwQ

양자화 표기 읽기

WbAcKVd는 가중치·activation·KV cache의 비트 수를 뜻한다

W 4 A 4 KV 4

W

Weights

A

Activations

KV

Key-Value cache

W4A4KV4: 세 요소를 모두 4비트 · AWQ W3: 가중치를 3비트로 압축 · BF16: 비교 기준인 full precision

다섯 개 벤치마크

짧은 산술부터 어려운 수학, 과학, 코드 생성까지

수학

- GSM8K · 짧은 산술
- MATH-500 · 다단계 문제
- AIME-120 · 매우 어려운 수학

과학

- GPQA-Diamond
- 대학원 수준 과학 QA

코딩

- LiveCodeBench
- 오염을 줄인 코드 평가

생성·평가 설정

모든 모델에 동일한 sampling 설정을 적용

0.6

temperature

0.95

top-p

CoT

평균 생성 토큰 수

- 정확도와 평균 CoT 길이를 함께 측정
- 오류 유형 분석은 Qwen-1.5B의 MATH-500 중심
- KL 분석은 동일한 prefix에서 BF16과 AWQ 3-bit 비교

결과 1: 정확도는 내려가고 생각 길이는 늘었다

강한 양자화일수록 낮은 성능과 긴 CoT가 동시에 나타났다

정확도 ↓

문제를 덜 맞힘

CoT 길이 ↑

더 많은 토큰 생성

비용 ↑

느리고 비싼 추론

압축으로 추론 비용이 줄어든 것이라는 기대와 반대

MATH-500 정확도

Qwen-1.5B에서 3-bit AWQ가 가장 크게 하락



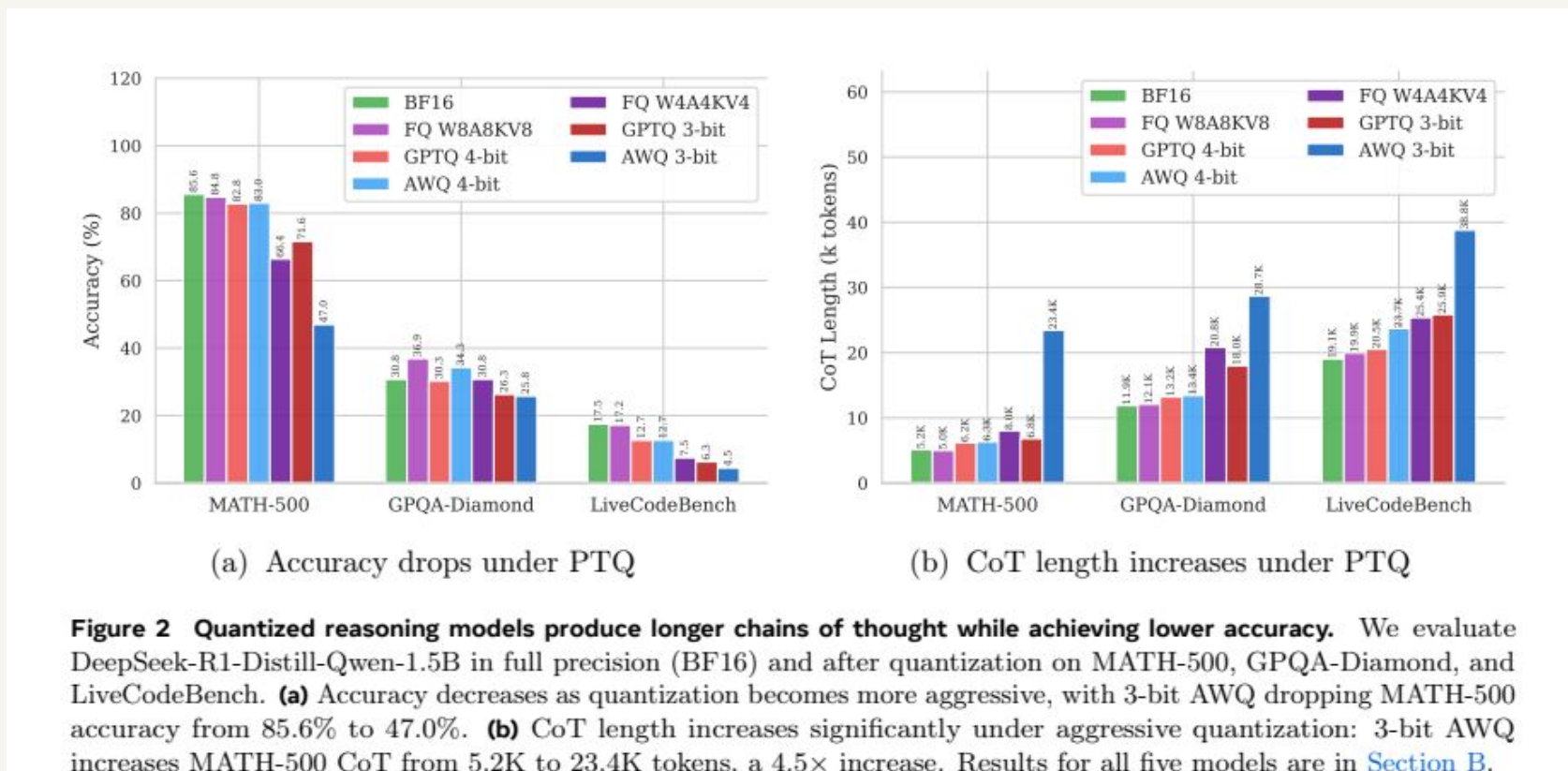
MATH-500 CoT 길이

AWQ 3-bit는 5.2K에서 23.4K 토큰으로 4.5배 증가



수학·과학·코딩에서 같은 방향

공격적 양자화가 세 영역에서 정확도와 효율을 함께 악화시켰다



원문 Figure 2

정확도 하락과 CoT 증가는 강하게 연결됐다

28개 모델-양자화 쌍의 Spearman $\rho = -0.73$

$$\rho = -0.73$$

강한 음의 순위 상관

- 정확도가 많이 떨어진 설정일수록 CoT가 더 길어짐
- 추가 추론이 단순한 부산물이 아닐 가능성
- 상관관계만으로 완전한 인과를 증명하지는 않음

오류를 네 종류로 분류

틀렸다는 결과만 보지 않고 어디서 잘못됐는지 구분

Overthinking

중간에 정답을 찾았지만 최종 답으로 확정하지 않음

논리 오류

초반 계획이나 문제 이해가 잘못됨

산술 오류

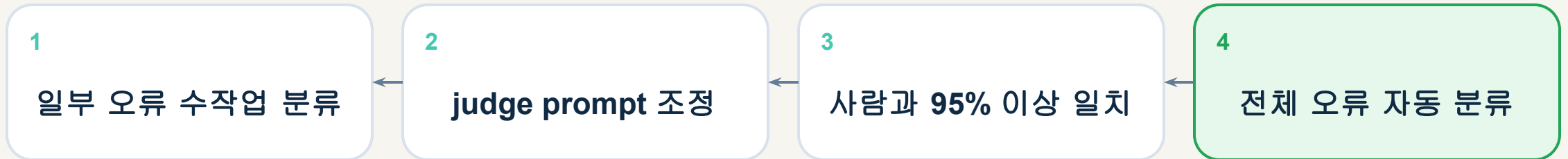
접근은 맞지만 계산에서 실수

형식·환각·기타

형식 위반, 없는 조건 추가, 기타 실패

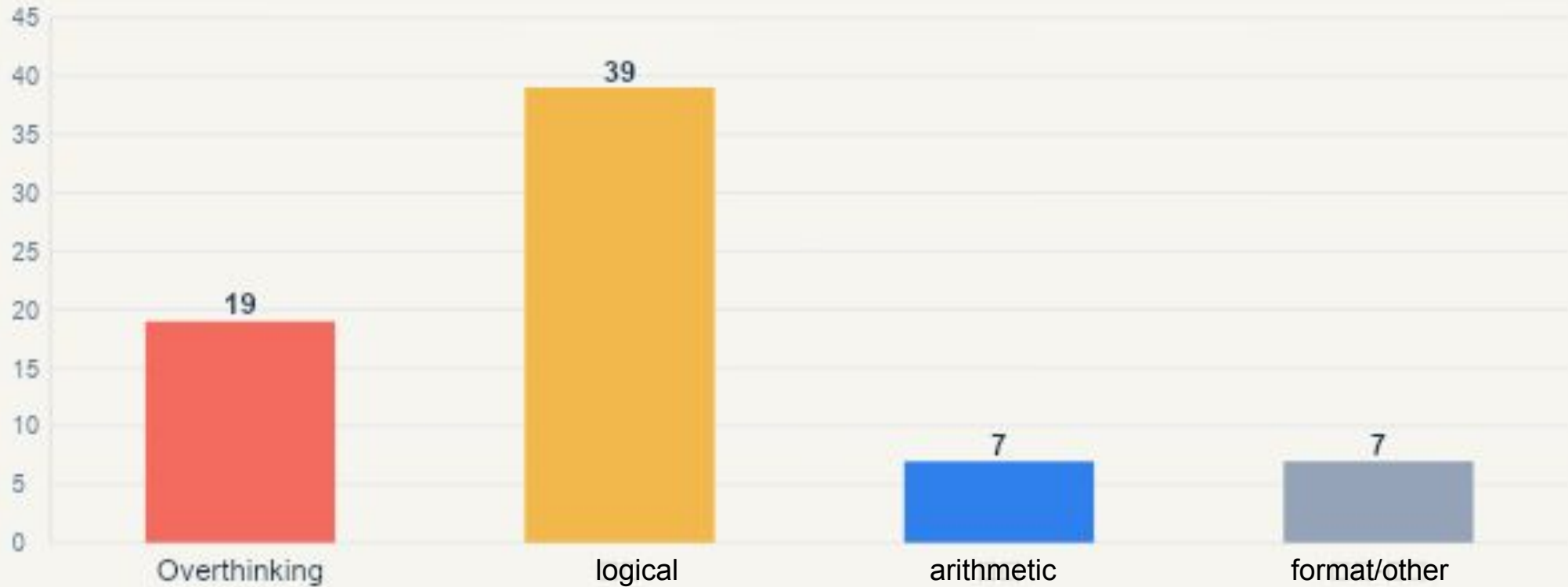
오류 분류 절차

사람의 표본 판정으로 프롬프트를 조정한 뒤 GPT-5 judge로 확장
데이터 검수 -> 클로드코드
생성형 요약 -> judge prompt 전체검수 (저품질,고품질)



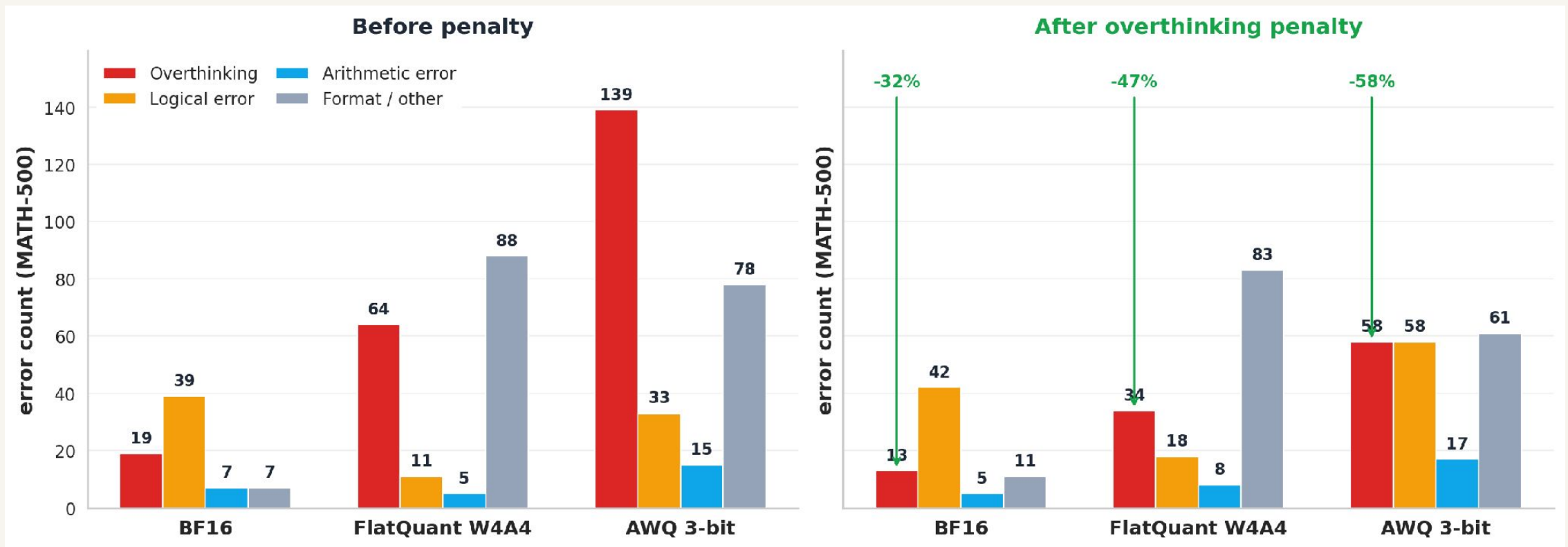
BF16도 overthinking하지만 비중은 낮았다

MATH-500에서 총 72개 오류 중 19개, 약 26%



AWQ 3-bit에서는 overthinking이 지배적 오류가 됐다

19개에서 139개로 7.3배 증가, 전체 오류의 52%



정답을 찾고도 버리는 실제 패턴

세 모델 모두 255에 도달했지만, 하나만 계속 새 가치를 열었다

BF16

- Answer: 255
- 약 5.2K tokens
- 정상 종료

AWQ 3-bit

- Answer: 255
- Wait... But... Alternatively...
- 약 23.4K, 계속 생성

+ penalty

- Answer: 255
- 가치치기
- 약 12.9K, 종료

중간 정답 255

그렇다면 어디서 분포가 흔들리는가

BF16과 양자화 모델이 같은 문맥에서 다음 토큰을 다르게 보는 지점을 찾는다

BF16
p(next token)

VS

AWQ 3-bit
q(next token)

- 같은 prompt와 같은 generation prefix 사용
- 각 위치에서 다음 토큰 분포 비교
- 차이가 큰 위치와 선택된 단어를 분석

KL divergence를 직관적으로 이해하기

두 모델의 '다음 단어 확률표'가 얼마나 다른지 재는 거리

P : BF16 원본 모델의 next-token 확률분포, Q : 양자화된 모델, 특히 AWQ 3-bit의 next-token 확률분포

$$D_{KL}(p \parallel q) = \sum_v p(v) \log[p(v) / q(v)]$$

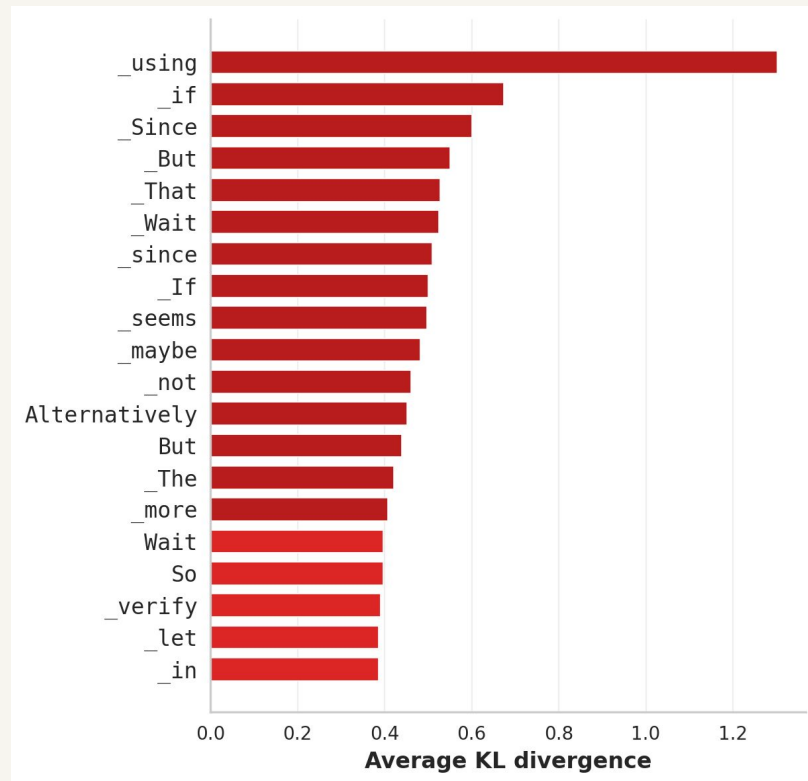
- [0.9, 0.05, 0.05] <-
[0.33, 0.33, 0.33] <-
- ~~KL~~ KL은 BF16과 양자화 모델의 예측이 어떻게 다를지
- 논문은 token position마다 KL을 계산

차이가 큰 토큰과 작은 토큰

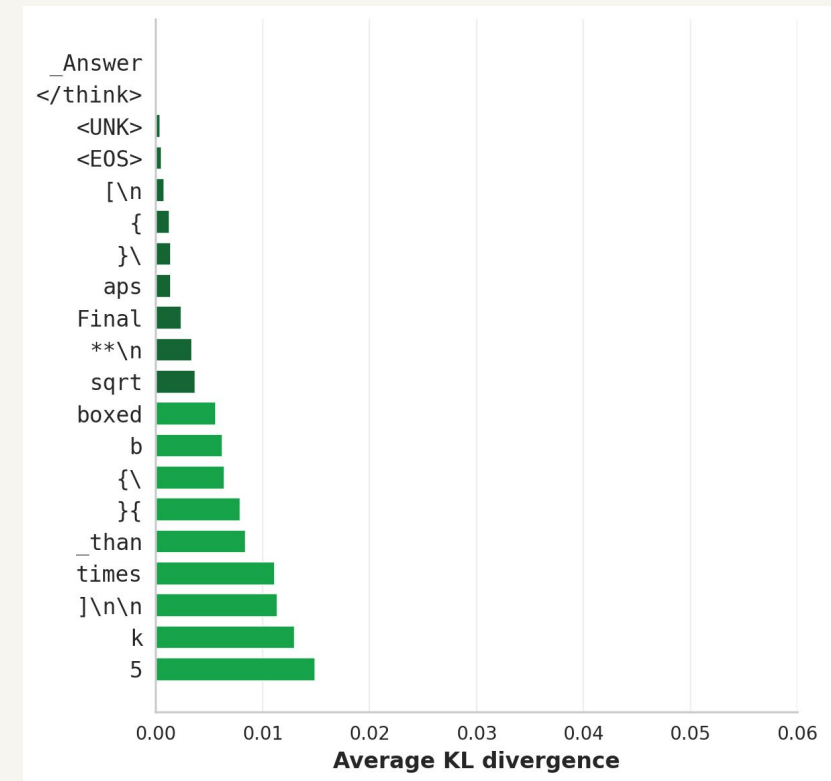
높은 KL에는 망설임·분기 단어, 낮은 KL에는 수학·형식 토큰이 많았다

낮은 KL = 양자화해도 원본 모델의 판단이 거의 유지됨

높은 KL = 양자화 때문에 원본 모델의 판단이 크게 흔들림



높은 KL · 망설임과 분기



낮은 KL · 계산과 형식

Entropy는 모델의 망설임 정도

한 토큰에 확률이 몰리면 낮고, 여러 후보에 퍼지면 높다

낮은 entropy

- 후보 하나가 압도적
- 계산의 다음 숫자·기호
- 작은 **perturbation**에도 안정적

확률이 한 후보에 집중

높은 entropy

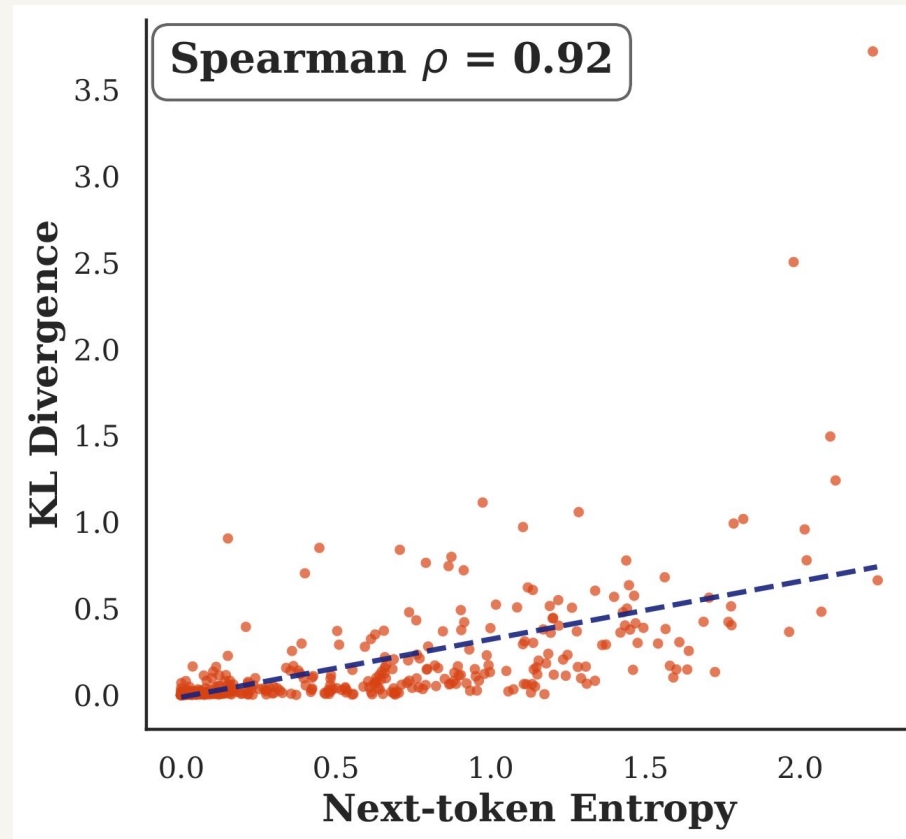
- 여러 후보가 비슷
- 분기 여부가 불확실
- 작은 **noise**가 선택을 바꿀 수 있음

확률이 여러 후보에 분산

불확실한 위치일수록 양자화의 영향이 컸다

위치별 KL과 BF16 next-token entropy의 Spearman $\rho = 0.92$

$p=[\text{라면:0.9, 김밥:0.05, 피자:0.05}]$ (엔트로피 낮음), $p=[\text{라면:0.3, 김밥:0.25, 피자:0.25, 햄버거:0.2}]$ (엔트로피 높음)



높은 entropy 위치에는 분기 단어가 더 가까이 있었다

overthinking marker가 top-20 후보에 들어올 확률이 2-4배 높았다

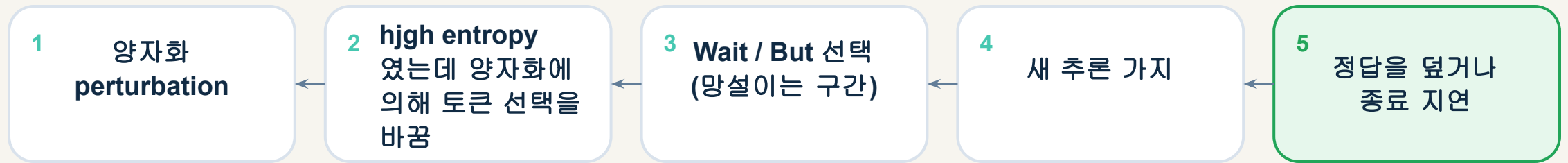
2-4×

top-20 후보 등장 빈도

- 'Wait', 'But' 같은 토큰이 선택될 문턱이 낮아짐
- 양자화 noise가 다음 토큰 순위를 뒤집기 쉬움
- 새 추론 가지가 열리면서 정답 확정이 지연됨

논문이 제안한 overthinking 메커니즘

양자화 오차가 불확실한 지점에서 분기 토큰을 밀어 올린다



양자화 perturbation : [5.02, 4.98, 1.2] 가 원 값인데 양자화에 의해 [4.95, 5.03, 1.18] 로 바뀌면서 교란이 생기는 것을 의미
low entropy vs high entropy : [0.98, 0.01, 0.01] vs [0.35, 0.34, 0.31]

해결책 : overthinking marker에 logit penalty

매 decoding step마다 50개 marker의 logit에서 고정값 λ 를 뺀다

$$z'_\square(v) = z_\square(v) - \lambda \quad \text{if } v \in S$$

- 재학습 불필요
- 추가 forward pass 불필요
- 조절 변수는 penalty strength λ 하나
- 모델을 다시 학습하는 대신, 추론 중 불필요한 재고찰이나 우회 경로를 시작하는 50개 marker token의 logit을 매 step에서 λ 만큼 낮춰 overthinking을 억제

50개 marker는 ‘생각의 새 가지’를 여는 단어

Wait · But · Alternatively · perhaps · maybe · reconsider · backtrack · wrong ...

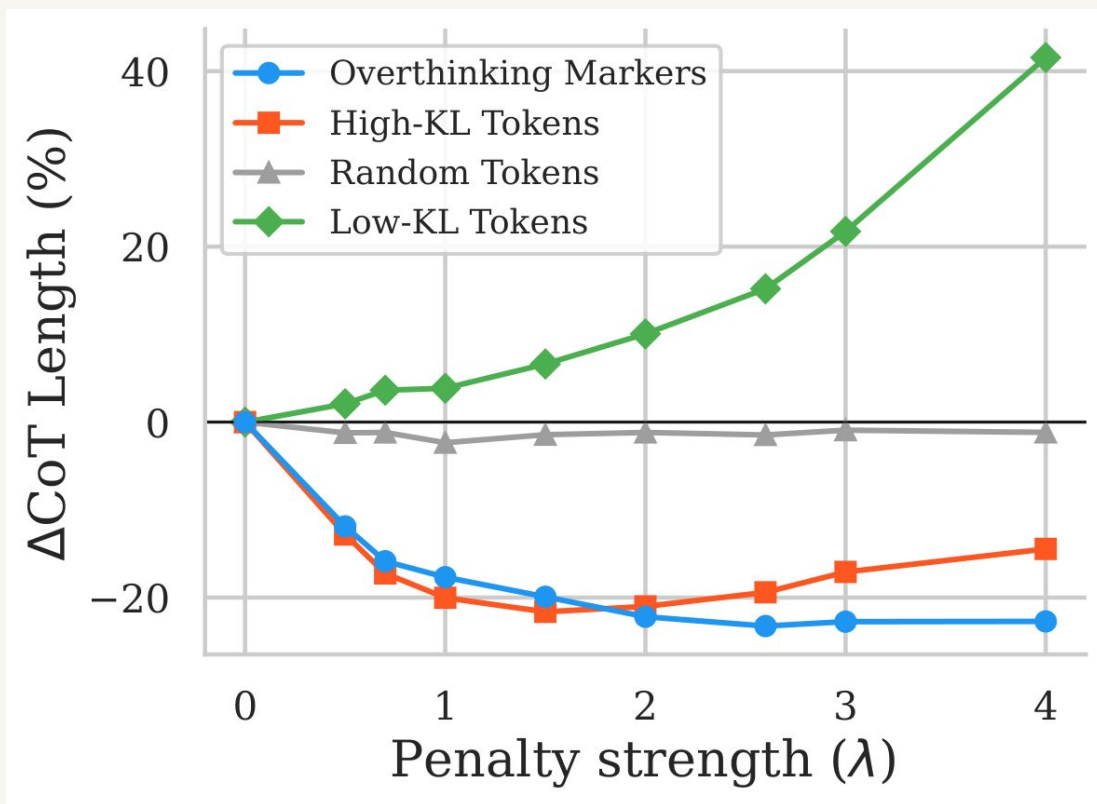
**Wait · But · Alternatively · perhaps · maybe · reconsider · backtrack ·
wrong ...**

- high-KL 목록 전체를 그대로 쓰지 않고 의미를 기준으로 선별
- ‘so’처럼 결론에 도움을 줄 수 있는 토큰은 제외
- 영어 추론 모델에 맞춰 사람이 직접 구성

절제 실험: 아무 토큰이나 누르면 같은가

같은 크기의 네 token set에 동일한 penalty를 적용 (qwen-1.5B, 6개 quantization 설정)

Low-KL token은 원본과 양자화 모델 모두 비교적 안정적으로 사용하는 토큰들입니다. -> 건들지 말자



Overthinking markers

- CoT 12–23% 감소
- 정확도 유지·소폭 개선 (High -KL overthinking marker 겹칠 수 있다)

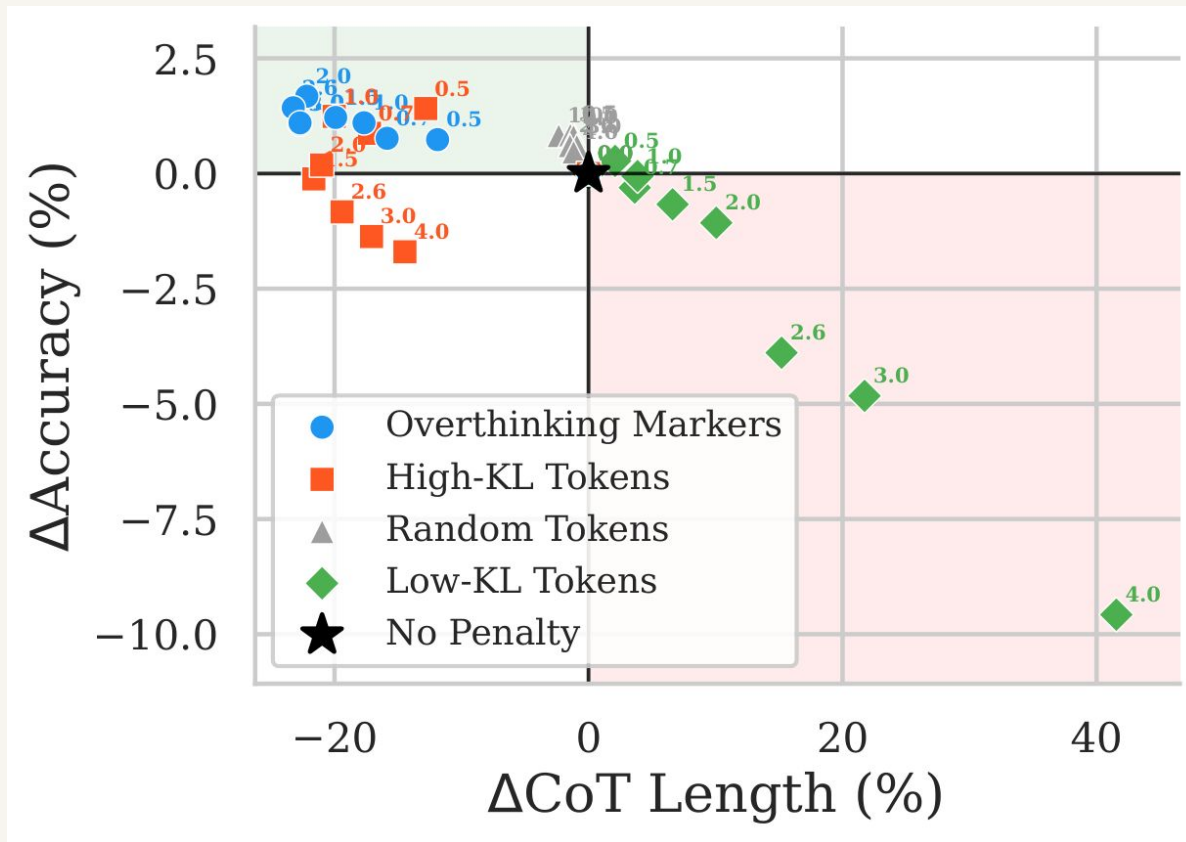
Low-KL tokens

- CoT 최대 41% 증가
- 정확도 최대 9.5%p 하락

효율과 정확도의 Pareto frontier가 개선됐다

COT Length와 accuracy의 파레토 관계

overthinking marker penalty만 짧은 CoT와 높은 정확도 쪽에 일관되게 위치



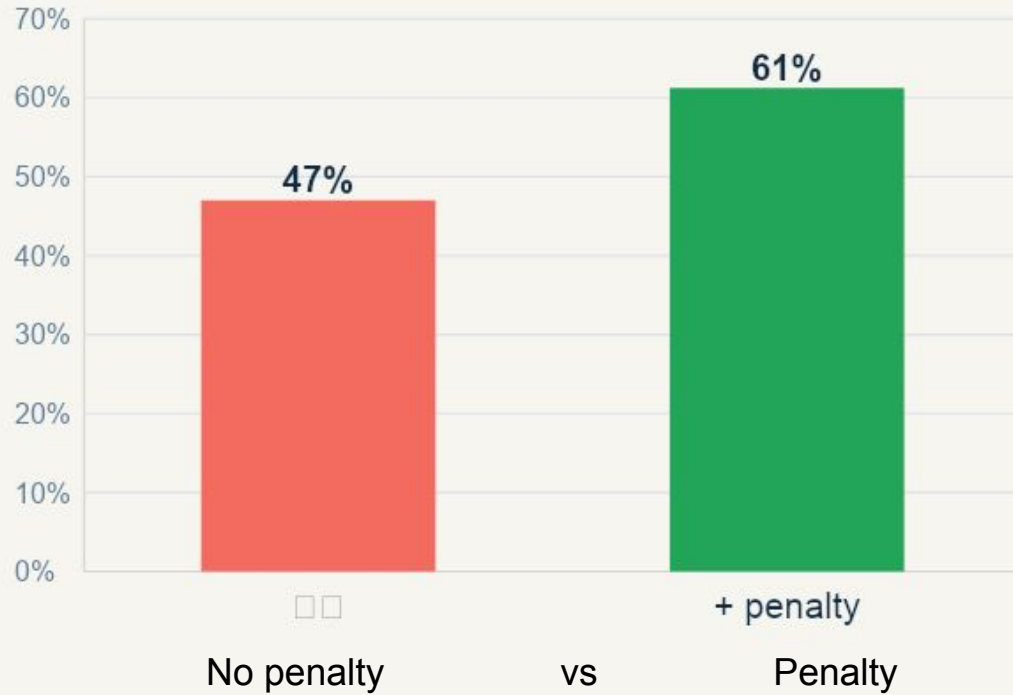
- random token penalty는 기준점 근처
- high-KL 전체를 강하게 누르면 정확도 손실
- negative λ 로 marker를 boost하면 CoT가 최대 445% 증가

대표 결과: AWQ 3-bit MATH-500

정답률은 14.2%p 높아지고 CoT는 약 45% 짧아졌다

정답률

Accuracy (%)



CoT length (K tokens)



어디까지 일반화되었고 무엇이 남았는가

다양한 설정에서 길이는 줄었지만, 해결책의 범위에는 경계가 있다

확인된 범위

- DeepSeek-R1-Distill-Qwen-1.5B
DeepSeek-R1-Distill-Qwen-7B
DeepSeek-R1-Distill-Qwen-14B
DeepSeek-R1-Distill-Llama-8B
QwQ-32B
- 3 PTQ methods
- 5 math/code/science benchmarks
- BF16도 CoT 5.6–10.0% 감소

남은 한계

- 고정 λ 만 사용
- 영어 marker를 수작업 구성
- 다른 planning domain 미검증
- arXiv preprint

한 장으로 정리

양자화는 모델의 '계산 능력'뿐 아니라 '언제 멈출지'도 바꾼다

- 1 강한 PTQ는 정확도를 낮추면서 CoT를 늘렸다
- 2 주요 실패는 정답을 찾고도 버리는 overthinking이었다
- 3 불확실한 위치의 분기 token을 약하게 누르자 비용과 오류가 함께 줄었다

토론 이 방법이 창의적 글쓰기에도 안전할까? / marker 대신 entropy 기반 동적 종료도 더 나올까?